

#중급반 2주차

그리디 알고리즘

<Greedy Algorithm>

2p /<그리디 알고리즘>

4p /<구현 방법>

8p /<동작 원리>

21p /<연습 문제>

#그리디 알고리즘

<Greedy Algorithm>

미래를 고려하지 않고, 매 순간 최선의 선택을 쌓아가며
결과적으로 최적의 답을 구해내는 알고리즘

단, 현재의 선택이 미래의 최적해 가능성을 배제하는 경우 사용할 수 없어요

#그리디 알고리즘

<Greedy Algorithm> - 구현 방법

1. 그리디하게 풀 수 있는 문제인지 확인한다.
2. 이 문제의 매 단계마다 할 수 있는 최선의 선택을 찾고, 예외 케이스가 있는지 살핀다.
3. 정답에 도달할 때까지 2번을 반복한다.

#그리디 알고리즘

<Greedy Algorithm> - 구현 방법

1. 그리디하게 접근할 수 있는 문제인지 확인한다.

그리디로 모든 문제를 풀 수 있는 건 아니에요.

문제를 부분 문제들로 나누어 풀어갈 수 있는 경우,

그리고 현재 단계에서의 최선의 선택이 전체의 최선의 선택으로 이어지는 경우에만 사용 가능해요.

2. 이 문제의 매 단계마다 할 수 있는 최선의 선택을 찾고, 예외 케이스가 있는지 살핀다.

현재의 선택이 다음 선택에 영향을 준다면 어떤 선택이 최선인지 알기 힘들어요.

당시에 최선으로 보이던 선택을 나중에 다시 바꾸어야 할 경우에는, 최선의 그리디한 전략이 아니에요.

3. 정답에 도달할 때까지 2번을 반복한다.

#그리디 알고리즘

<Greedy Algorithm> - 연습 문제

2 거스름돈 (BOJ #5585)

<문제 설명>

- 잔돈으로 쓰일 1, 5, 10, 50, 100, 500원짜리 동전이 무한히 많이 있는 가게가 있다.
- 물건 n 원어치를 사고, 1000원을 주었다.
- 동전의 개수가 최소가 되도록 거슬러 받는다.
- 동전이 몇 개 있을까?

<제약 조건>

- $1 \leq n < 1,000$

#그리디 알고리즘

<Greedy Algorithm> - 연습 문제

2 거스름돈 (BOJ #5585)

<코드 설명>

- 가치가 큰 것부터 교환하면 될 거 같고, 실제로 그렇게 구현하면 됩니다.

```
int main(){
    int cost;
    cin >> cost;

    int change = 1000 - cost;
    int count = 0;

    // 사용할 수 있는 동전 단위 (큰 순서대로)
    int coins[6] = {500, 100, 50, 10, 5, 1};

    // 매 단계에서 가장 가치가 높은 동전부터 '그리디하게' 선택
    for (int i = 0; i < 6; i++) {
        if (change == 0) break; // 거스름돈을 다 줬다면 종료

        // 현재 동전으로 줄 수 있는 최대 개수를 구함
        count += (change / coins[i]);

        // 남은 거스름돈 업데이트 (나머지 연산)
        change %= coins[i];
    }

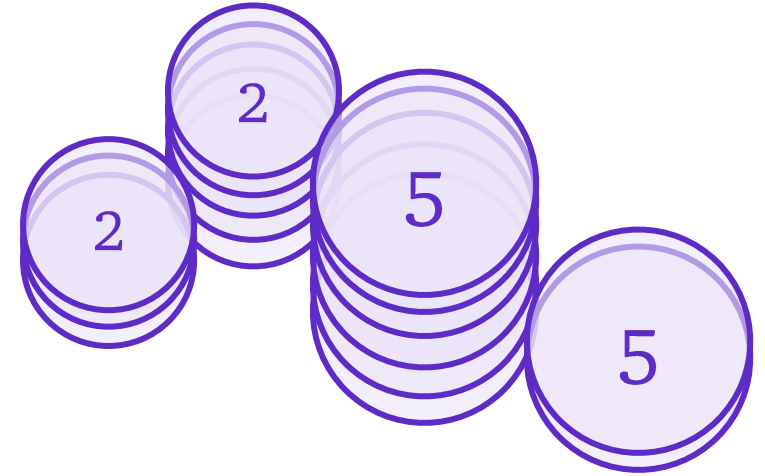
    // 결과 출력
    cout << count << endl;
}
```

#그리디 알고리즘

<Greedy Algorithm> - 동작 원리

<예시 문제>

2원짜리 동전과 5원짜리 동전만을 사용해서 거슬러주어야 할 돈 n 원이 주어졌을 때 거슬러주어야 하는 동전의 **최소 개수**를 구하여라.
(단, 각 동전들의 개수는 충분히 많다고 가정한다.)



#그리디 알고리즘

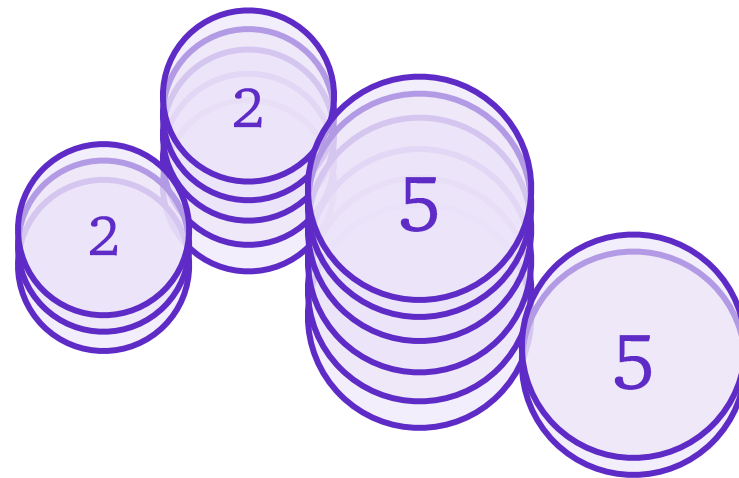
<Greedy Algorithm> - 동작 원리

<예시 문제>

2원짜리 동전과 5원짜리 동전만을 사용해서 거슬러주어야 할 돈 n 원이 주어졌을 때 거슬러주어야 하는 동전의 **최소 개수**를 구하여라.
(단, 각 동전들의 개수는 충분히 많다고 가정한다.)

<풀이>

1. 직관적으로 생각했을 때, 5원짜리 동전을 많이 쓸수록 동전의 개수가 적어질 것이다.
2. 그렇다면 가능한 한 적게 2원짜리 동전을 사용해보고, 5원짜리 동전을 최대한 많이 써 볼까?



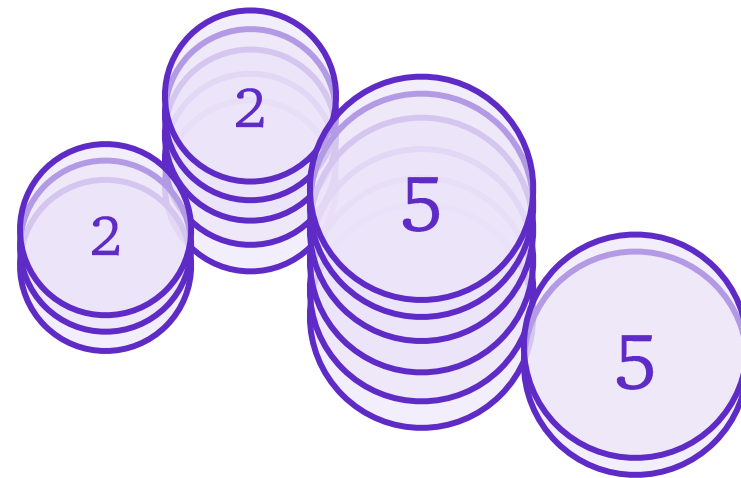
#그리디 알고리즘

<Greedy Algorithm> - 동작 원리

<예시 문제>

2원짜리 동전과 5원짜리 동전만을 사용해서 거슬러주어야 할 돈 n 원이 주어졌을 때 거슬러주어야 하는 동전의 **최소 개수**를 구하여라.

(단, 각 동전들의 개수는 충분히 많다고 가정한다.)



<풀이>

1. 직관적으로 생각했을 때, 5원짜리 동전을 많이 쓸수록 동전의 개수가 적어질 것이다.
2. 그렇다면 가능한 한 적게 2원짜리 동전을 사용해보고, 5원짜리 동전을 최대한 많이 써 볼까?
 -> 최선의 선택을 반복하여 정답을 도출할 수 있다!
 ('5원짜리 동전을 사용할 수 없으면 2원짜리 동전을 쓰고, 사용할 수 있으면 5원 동전으로 전부 처리한다.'를 반복)

#그리디 알고리즘

<Greedy Algorithm> - 연습 문제

5 거스름돈 (BOJ #14916)

<문제 설명>

- 거슬러주어야 할 돈 n 원이 주어진다.
- 2원짜리 동전과 5원짜리 동전이 무한히 많이 주어진다.
- 동전의 개수가 최소가 되도록 거슬러 주어야 한다.
- 동전을 거슬러 줄 수 없다면 -1을 출력한다.

<제약 조건>

- $1 \leq n \leq 100,000$

#그리디 알고리즘

<Greedy Algorithm> - 연습 문제

5 거스름돈 (BOJ #14916)

<문제 해설>

“가능한 한 많은 5원짜리 동전을 사용하고, 가능한 한 적은 2원짜리 동전을 사용하자.”

만약 거스름돈이 15원이라면, $(15 / 5 = 3)$ 5원짜리 동전 3개를 사용하면 동전의 개수를 최소화할 수 있다.

만약 거스름돈이 14원이라면, $(14 / 5 = 2)$ 5원짜리 동전 2개와 2원짜리 동전 2개를 사용하면 된다.

#그리디 알고리즘

<Greedy Algorithm> - 연습 문제

5 거스름돈 (BOJ #14916)

<문제 해설>

“가능한 한 많은 5원짜리 동전을 사용하고, 가능한 한 적은 2원짜리 동전을 사용하자.”

만약 거스름돈이 15원이라면, $(15 / 5 = 3)$ 5원짜리 동전 3개를 사용하면 동전의 개수를 최소화할 수 있다.

만약 거스름돈이 14원이라면, $(14 / 5 = 2)$ 5원짜리 동전 2개와 2원짜리 동전 2개를 사용하면 된다.

그런데 만약, 거스름돈이 1원이라면? 3원이라면?

#그리디 알고리즘

<Greedy Algorithm> - 구현 방법

1. 그리디하게 풀 수 있는 문제인지 확인한다.
2. 이 문제의 매 단계마다 할 수 있는 최선의 선택을 찾고, 예외 케이스가 있는지 살핀다.
3. 정답에 다다를 때까지 2번을 반복한다.

#그리디 알고리즘

<Greedy Algorithm> - 구현 코드

5 거스름돈 (BOJ #14916)

예외 케이스는 따로!

그리디 알고리즘은 앞에서부터/뒤에서부터
순차적으로!

```
int main(){
    int charge, answer = 0;
    cin >> charge;

    // 예외 처리
    if(charge == 1 || charge == 3){
        cout << -1 << '\n';
    }
    else{
        // 5로 나누어 떨어질 때까지 2씩 빼주며 카운트
        while(charge > 0){
            if(charge % 5 == 0){ // 5로 나누어 떨어질 때
                answer += charge / 5;
                break;
            }
            else{ // 5로 나누어 떨어지지 않을 때
                charge -= 2;
                answer++;
            }
        }
        cout << answer << '\n';
    }
}
```

#필수 문제

2 거스름돈 (BOJ #5585)

앞에서 설명한 문제 1

4 동전 0 (BOJ #11047)

N종류의 동전으로 K원을 만드는 문제

5 거스름돈 (BOJ #14916)

앞에서 설명한 문제 2

4 설탕 배달 (BOJ #2839)

동전의 종류가 '5원'과 '3원', 2종류일 땐 어떻게 될까요?

#연습 문제 도전

4 ATM (BOJ #11399)

어떤 전략이 최선일까요? (Hint: 정렬)

2 $A_i \times A_j$ (BOJ #32630)

수학적인 발상이 조금은 필요한 문제

5 회의실 배정 (BOJ #1931)

진짜 정말 많이 유명한 문제

3 주유소 (BOJ #13305)

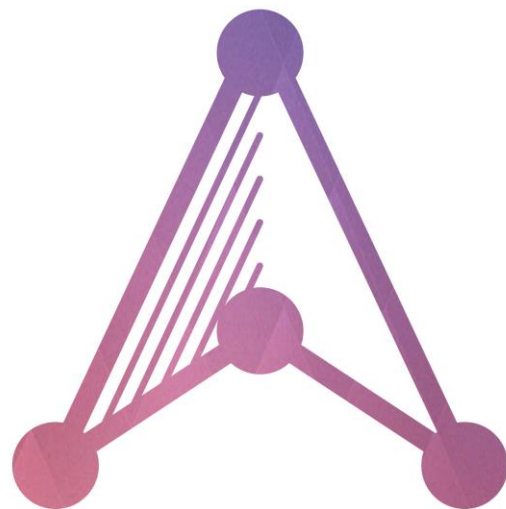
어떤 전략으로 기름을 넣어야 돈을 아낄 수 있을까요?

2 Pipelined (BOJ #30874)

2023 HCPC Beginner G번

4 잘 정의된 들여쓰기 (BOJ #35309)

코딩테스트 스타일의 문제



A L O H A
The algorithm club.